

# The Art of Computer Programming 3

## Lecture Notes

|             |                                       |    |
|-------------|---------------------------------------|----|
| Lecture 1.  | Computable and uncomputable functions | 2  |
| Lecture 2.  | Decidable and enumerable languages    | 6  |
| Appendix A. | Proof by cases                        | 12 |

# Lecture 1. Computable and uncomputable functions

## 1.1 Alphabets and words

At the low level, information inside computers is represented by 0s and 1s. Every program takes as input a finite sequence of bits and returns another such sequence, and all computation as we know it, in the end, is manipulation of bit-strings. Our formalisation begins here.

**Definition 1** (Alphabets and words). An **alphabet**  $\Sigma$  is a non-empty finite set of symbols. A **word** (or **string**) over  $\Sigma$  is a finite sequence of symbols  $w = a_1 a_2 \dots a_n$ , where each  $a_i \in \Sigma$ . The unique sequence of length zero is the **empty word**, denoted  $\varepsilon$ . The set of all finite words over  $\Sigma$  is denoted  $\Sigma^*$ .

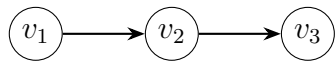
Given two words  $u = a_1 \dots a_n$  and  $v = b_1 \dots b_m$ , their **concatenation** is  $uv = a_1 \dots a_n b_1 \dots b_m$ . Concatenation is associative:  $(uv)w = u(vw)$ , and  $\varepsilon$  is the two-sided identity:  $u\varepsilon = \varepsilon u = u$ .<sup>1</sup>

**Binary encoding.** The ‘canonical’ alphabet for computers is the *binary alphabet*  $\{0, 1\}$ . Words in any other alphabet  $\Sigma$  can be *encoded* by binary words as follows. We fix a length  $k$  such that  $2^k \geq |\Sigma|$ , and assign a distinct  $k$ -bit binary word to each symbol  $a_i$ . For example, if  $\Sigma = \{a, b, c\}$ , we can choose  $k = 2$  and map

$$a \mapsto 00, \quad b \mapsto 01, \quad c \mapsto 10.$$

A word over  $\Sigma$  is then encoded by replacing each symbol with its code: for instance,  $cab$  becomes 100001. Since every symbol uses exactly 2 bits (or, in general,  $k$ ), decoding is unambiguous: we simply read the string two bits at a time.<sup>2</sup>

Other finite objects that programs work with are encoded similarly. For example, a directed graph  $G$  on vertices  $\{v_1, \dots, v_n\}$  can be represented by its  $n \times n$  adjacency matrix  $A$ , where  $A_{i,j} = 1$  if there is an edge  $v_i \rightarrow v_j$  and 0 otherwise. Flattening this matrix row by row yields a binary word of length  $n^2$ . For instance, the graph on 3 vertices with edges  $v_1 \rightarrow v_2$  and  $v_2 \rightarrow v_3$  and its corresponding adjacency matrix are:



$$A = \begin{pmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{pmatrix},$$

which is encoded by the word 010001000.

## 1.2 Computable functions

Fix your favourite general-purpose programming language (we will use Python in these notes). Consider programs that take one input word and produce one output word over some alphabet  $\Sigma$ . As explained above, multiple arguments, numbers, graphs, or structured data can always be encoded into a single word in  $\Sigma^*$  (and decoded from the output).

<sup>1</sup>If you like algebra, you may note that  $\Sigma^*$  with concatenation and  $\varepsilon$  is thus the free monoid generated by  $\Sigma$ .

<sup>2</sup>In ASCII, a standard encoding in real computers, each character is an 8-bit byte.

Thus, each program  $\pi$  naturally computes a (partial!) function  $f_\pi: \Sigma^* \rightharpoonup \Sigma^*$ .<sup>3</sup> This is how it is defined.

**Definition 2** (Function computed by a program  $\pi$ ). *Given  $u \in \Sigma^*$ , run  $\pi$  on  $u$ .*

- *If  $\pi$  halts (i.e., terminates, concludes the execution), then  $f_\pi(u) = v$ , where  $v \in \Sigma^*$  is the word it outputs.*
- *If  $\pi$  does not halt, then  $f_\pi(u)$  is undefined.*

In real life, programs may also crash with an error. In theory, we treat crashes as equivalent to non-halting.

**Definition 3** (Computable function). *A partial function  $f: \Sigma^* \rightharpoonup \Sigma^*$  is called **computable** if there exists a program  $\pi$  such that  $f = f_\pi$ .*

Note that when  $f: \Sigma^* \rightarrow \Sigma^*$  is **total**, i.e., defined on all words in  $\Sigma^*$ , the corresponding program  $\pi$  must halt on every input.

Let us consider some examples of computable functions.

**Example 4.** The following functions are all computable:

- *reverse*: given  $w \in \Sigma^*$ , outputs its reverse (e.g.,  $\text{reverse}(abc) = cba$ );
- *convert*: interprets  $w \in \{0, 1, \dots, 9\}^*$  as an integer  $n$ , outputs a word in  $\{0, 1, 2, 3\}^*$ —the same number, but in 4-ary notation;
- *double*: interprets  $w$  as an integer  $n$ , outputs  $2n$ ;
- *multiply*: given a pair of integers  $\langle n, k \rangle$ , outputs their product  $n \cdot k$ ;
- *shortestPath*: given a directed graph and two vertices  $\langle G, s, t \rangle$ , outputs the length of the shortest path from  $s$  to  $t$  (undefined if no path exists);
- *deanon*: given  $w$ , if  $w$  encodes the real name of Banksy,  $\text{deanon}(w)$  encodes the real name of Satoshi Nakamoto; for all other  $w$ , *deanon* is undefined.

### 1.3 Uncomputable functions

Are there any functions that are not computable? Yes, and it is easy to prove: there are only countably many programs, but uncountably many functions on  $\{0, 1\}^*$ . Let us see this argument in detail.

---

<sup>3</sup>Note the arrow! It is  $\rightharpoonup$ , not  $\rightarrow$ . This is how we denote that the function is *partial*.

Recall that a set  $S$  is called *countable* if its elements can be indexed by natural numbers. The set  $\{0, 1\}^*$  is, of course, countable—here is a possible indexing of words in  $\{0, 1\}^*$ :

$$\begin{aligned}
 w_0 &= \varepsilon \\
 w_1 &= 0 \\
 w_2 &= 1 \\
 w_3 &= 00 \\
 w_4 &= 01 \\
 w_5 &= 10 \\
 w_6 &= 11 \\
 w_7 &= 000 \\
 &\dots
 \end{aligned} \tag{1}$$

That is, we list the words by increasing length, and order words of the same length lexicographically. Similarly,  $\Sigma^*$  is countable for any alphabet  $\Sigma$ .

**Programs are words themselves.** Indeed, programs are just sequences of symbols that we write, and thus can be seen as words. In fact, the converse is also true: we can try to execute any word as if it was a program and see what happens—a crash would just mean non-halting. So every program is a word and every word encodes some (maybe nonsensical) program. The set of all programs, and hence of all computable functions, is therefore countable.

In contrast, the set of all partial functions  $\{0, 1\}^* \rightarrow \{0, 1\}^*$  is uncountable. In fact, this is already true for the set of all functions  $\{0, 1\}^* \rightarrow \{0, 1\}$ .

**Proposition 5** (Cantor, 1891). *The set of all functions  $\{0, 1\}^* \rightarrow \{0, 1\}$  is uncountable.*

*Proof.* For clarity of notation, we use the correspondence (1) and consider the set of functions  $\mathbb{N} \rightarrow \{0, 1\}$ . Assume for contradiction that this set is countable. Then we can index all functions in this set:  $f_0, f_1, f_2, \dots$ . Each function  $f_i$  is defined by its values on each  $k \in \mathbb{N}$ . Let us write them out:

|                            |                            |                            |                            |         |
|----------------------------|----------------------------|----------------------------|----------------------------|---------|
| <b><math>f_0(0)</math></b> | $f_0(1)$                   | $f_0(2)$                   | $f_0(3)$                   | $\dots$ |
| $f_1(0)$                   | <b><math>f_1(1)</math></b> | $f_1(2)$                   | $f_1(3)$                   | $\dots$ |
| $f_2(0)$                   | $f_2(1)$                   | <b><math>f_2(2)</math></b> | $f_2(3)$                   | $\dots$ |
| $f_3(0)$                   | $f_3(1)$                   | $f_3(2)$                   | <b><math>f_3(3)</math></b> | $\dots$ |
| $\dots$                    |                            |                            |                            |         |

Consider the diagonal of this infinite table, in bold, and the function:

$$d(n) = \begin{cases} 1, & \text{if } f_n(n) = 0, \\ 0, & \text{if } f_n(n) = 1. \end{cases}$$

Then,  $d$  is a function  $\mathbb{N} \rightarrow \{0, 1\}$ , so it must appear in the above list  $f_0, f_1, f_2, \dots$  of all such functions. However, by construction,  $d$  disagrees with  $f_0$  on 0, with  $f_1$  on 1, and, in general, with  $f_n$  on  $n$ . A contradiction.  $\square$

**Corollary 6.** *There exist uncomputable functions from  $\{0,1\}^*$  to  $\{0,1\}$ .*

In fact, there are uncountably many.

And so what? No concrete example was given so far. Could it be that all *useful* functions that we may ever want to compute are countable? Unfortunately, not. Consider the following very useful *halting function*:

$$h(\langle \pi, w \rangle) = \begin{cases} 1, & \text{if } \pi \text{ halts on input } w, \\ 0, & \text{if } \pi \text{ loops forever on input } w. \end{cases}$$

Its argument is a *pair*: a program  $\pi$  and a word  $w$ , put in the  $\langle \rangle$ -brackets and separated by a comma—this is just a way to ‘pack’ two different things into one argument of a function. We assume that  $\langle, \rangle$  and the comma are all symbols of  $\Sigma$  that are not used in programs and input words.<sup>4</sup> Note that  $h$  is a total function.

**Theorem 7** (Turing, 1936). *The halting predicate is (alas!) uncomputable.*

*Proof.* Recall that we have an indexing of words in  $\Sigma^*$ :  $w_0, w_1, w_2, \dots$ , each of which can be seen as a program:  $\pi_0, \pi_1, \pi_2, \dots$ . Each  $\pi_i$  can (potentially) be run on each  $w_j$  (if  $i = j$ , we run  $\pi_i$  on its own code!)

We can imagine a table of results for the halting predicate:

|          | $w_0$                             | $w_1$                             | $w_2$                             | $w_3$                             | $\dots$  |
|----------|-----------------------------------|-----------------------------------|-----------------------------------|-----------------------------------|----------|
| $\pi_0$  | <b><math>h(\pi_0, w_0)</math></b> | $h(\pi_0, w_1)$                   | $h(\pi_0, w_2)$                   | $h(\pi_0, w_3)$                   | $\dots$  |
| $\pi_1$  | $h(\pi_1, w_0)$                   | <b><math>h(\pi_1, w_1)</math></b> | $h(\pi_1, w_2)$                   | $h(\pi_1, w_3)$                   | $\dots$  |
| $\pi_2$  | $h(\pi_2, w_0)$                   | $h(\pi_2, w_1)$                   | <b><math>h(\pi_2, w_2)</math></b> | $h(\pi_2, w_3)$                   | $\dots$  |
| $\pi_3$  | $h(\pi_3, w_0)$                   | $h(\pi_3, w_1)$                   | $h(\pi_3, w_2)$                   | <b><math>h(\pi_3, w_3)</math></b> | $\dots$  |
| $\vdots$ | $\vdots$                          | $\vdots$                          | $\vdots$                          | $\vdots$                          | $\ddots$ |

We are going to mess with the diagonal again! Suppose for contradiction that  $h$  is computed by some program  $\eta$ . We define the following *diagonal* program  $\delta$ :

```
def  $\delta(w)$ :
    if  $\eta(w, w) == 1$ :
        while True: pass    # loop forever
    else:
        return 0             # halt immediately
```

Since  $\delta$  is a valid program, we could ask what happens when  $\delta$  is run on its own code. If  $\delta$  halts when run on the word encoding  $\delta$ , then  $\eta(\delta, \delta)$  must return 1, and so  $\delta$ , according to its own code, should actually loop forever. If it does not halt,  $\eta(\delta, \delta)$  returns 0, and then  $\delta$  should halt. A contradiction.  $\square$

<sup>4</sup>This creates a problem: we need to define  $h$  also on ‘malformed’ inputs—such as  $\langle \langle, \rangle, \rangle$ —which is formally a word from  $\Sigma^*$ . But we can agree that  $h$  will be equal to 0 on such words and forget about them. What we really care about is the behaviour of  $h$  on properly formed words: a  $\langle$ , then a program, then a comma, then an input word, then a  $\rangle$ .

## Lecture 2. Decidable and enumerable languages

### 2.1 From functions to languages

Let  $f: \Sigma^* \rightarrow \{0, 1\}$  be a total function that outputs a single bit. It can be thought of as classifying input words into two categories: those on which  $f$  returns 1 (the ‘yes’ words) and those on which  $f$  returns 0 (the ‘no’ words).

This is the same as specifying a subset  $L \subseteq \Sigma^*$ —namely, the set  $L = \{w \in \Sigma^* \mid f(w) = 1\}$ .

Conversely, every set  $L \subseteq \Sigma^*$  defines a one-bit function, called its **characteristic function**:

$$\chi_L(w) = \begin{cases} 1, & \text{if } w \in L, \\ 0, & \text{if } w \notin L. \end{cases}$$

Every set  $L \subseteq \Sigma^*$  is associated with the following problem of binary classification, called the **decision problem**<sup>5</sup> for  $L$ :

*Input:*  $w \in \Sigma^*$

*Question:* Does  $w$  belong to  $L$ ?

**Definition 8.** A set of words  $L \subseteq \Sigma^*$  is called a **language**. It is **decidable** when its characteristic function is computable, and **undecidable** otherwise.

A program that computes the characteristic function  $\chi_L$  is said to **decide** the language  $L$ : it outputs 1 on words from  $L$  (accepts them) and 0 on words not from  $L$  (rejects them). Note that such a program must halt on every input—and give an answer.

We thus have a correspondence between one-bit functions and languages:

|                                           | Function $f: \Sigma^* \rightarrow \{0, 1\}$ | Language $L \subseteq \Sigma^*$      |
|-------------------------------------------|---------------------------------------------|--------------------------------------|
| <b>Task (‘problem’)</b>                   | given $w$ , compute $f(w)$                  | given $w$ , decide whether $w \in L$ |
| <b>If some <math>\pi</math> does that</b> | computable                                  | decidable                            |
| <b>This <math>\pi</math></b>              | computes the function                       | decides the language                 |

**Example 9.** The empty language  $L = \emptyset$  is decidable. It is decided by the following program:

```
def reject_all(w):
    return 0
```

**Example 10.** Any finite language  $L = \{w_0, w_1, \dots, w_n\}$  is decidable:

<sup>5</sup>The term is historical: basically, we are asked to ‘decide’ if the input word belongs to the set.

```
def decide_L(w):
    words_L = [w0, w1, ... , wn]
    if w in words_L:
        return 1
    else:
        return 0
```

**Example 11.** The language

$$\text{PRIMES} = \{w \in \{0,1\}^* \mid w \text{ is the binary encoding of a prime number}\}$$

is decidable:

```
def is_prime(w):
    n = int(w, 2)          # interpret binary string as integer
    if n < 2:
        return 0
    for d in range(2, n):
        if n % d == 0:
            return 0
    return 1
```

**Example 12.** The language

$$\text{HALT} = \{\langle \pi, w \rangle \mid \pi \text{ halts on input } w\}$$

is undecidable, by Theorem 7.

It is a tradition in the theory of computing to name languages in ALL CAPS. It might be slightly ANNOYING, but that's the tradition, and we will RESPECT IT.

## 2.2 Semantic properties of programs

We have seen that HALT is undecidable, which is sad for those who would like to *verify* that their programs behave in the expected manner—at least, halt on a given word. Are there properties of programs that one can effectively check?

Of course there are. For example, this one:

*Input:* program code

*Question:* Does this program use a variable named  $x$ ?

It should be clear that the property above is qualitatively different from HALT. The former asks about the program's *syntax*, the latter—about the *function* that the program computes, i.e. the program's *semantics*. To highlight this distinction, we introduce some terminology.

Recall that every  $w \in \Sigma^*$  can be treated as a program: feed it to Python's interpreter, specify an input, and it will do something—maybe crash immediately, maybe output Banksy's real name, or maybe output the input backwards...—it will compute some partial function  $\Sigma^* \rightarrow \Sigma^*$ .

Any language  $P \subseteq \Sigma^*$  defines a **property of programs**: a program  $\pi$  has property  $P$  if  $\pi \in P$ , and does not have it if  $\pi \notin P$ .

**Definition 13.** A property of programs  $P \subseteq \Sigma^*$  is called **semantic** if for any two programs  $\pi_1, \pi_2 \in \Sigma^*$ , if  $f_{\pi_1} = f_{\pi_2}$ , then either both  $\pi_1, \pi_2 \in P$  or both  $\pi_1, \pi_2 \notin P$ .

In other words, a semantic property is a property of *functions*: for every function, either all programs that compute it have the property, or none does.

A property  $P$  is **non-trivial** if  $P \neq \emptyset$  and  $P \neq \Sigma^*$ . That is, there are some programs that have it, and some that do not.

**Example 14.** Consider the property

$$\text{HALT-ON-EMPTY} = \{\pi \mid \pi \text{ halts on input } \varepsilon\}.$$

It is semantic:  $\pi \in \text{HALT-ON-EMPTY}$  iff  $f_\pi(\varepsilon)$  is defined. It is non-trivial: a program that always outputs 1 regardless of the input has this property, and one that loops forever on all inputs does not have it.

The following theorem is the saddest theorem in our course.

**Theorem 15** (Rice, 1953). *Every non-trivial semantic property of programs is undecidable.*

*Proof.* The proof was not covered in the lecture, so it is here just for the curious.

Let  $P \subseteq \Sigma^*$  be a non-trivial semantic property, and consider the following program:

```
def never_stop(w):
    while True: pass    # loops forever regardless of the input
```

The function that it computes,  $f_{\text{never\_stop}}$ , is the (unique) function that is undefined on all inputs.

Suppose that  $\text{never\_stop} \notin P$ . (If  $\text{never\_stop} \in P$ , consider the complement  $\bar{P} = \Sigma^* \setminus P$  instead, which is also a non-trivial semantic property. Properties  $P$  and  $\bar{P}$  are either both decidable or both undecidable, so this is without loss of generality.)

Since  $P$  is non-empty, there is some program  $\sigma \in P$ . Since  $P$  is semantic and  $\text{never\_stop} \notin P$ , we have that  $f_\sigma \neq f_{\text{never\_stop}}$ , meaning that  $\sigma$  halts on at least one input.

We reduce the Halting Problem to  $P$ . Given a pair  $\langle \pi, w \rangle$ , construct the following program  $\rho_{\pi, w}$ :

```
def rho_pi_w(x):
    pi(w)                # run pi on w (ignore x)
    return sigma(x)      # if pi halted, behave like sigma
```

If  $\pi$  halts on  $w$ , then  $\rho_{\pi,w}$  behaves exactly like  $\sigma$ , so  $f_{\rho_{\pi,w}} = f_\sigma$  and  $\rho_{\pi,w} \in P$ . If  $\pi$  does not halt on  $w$ , then  $\rho_{\pi,w}$  never reaches the second line, so  $f_{\rho_{\pi,w}} = f_{\text{never\_stop}}$  and  $\rho_{\pi,w} \notin P$ .

Therefore,  $\langle \pi, w \rangle \in \text{HALT}$  if and only if  $\rho_{\pi,w} \in P$ . If  $P$  were decidable by some program `decide_P`, we would be able to decide HALT as follows:

```
def decide_halt( $\pi$ , w):
     $\rho_{\pi,w}$  = construct( $\pi$ , w)    # construct the code of  $\rho_{\pi,w}$  as above
    return decide_P( $\rho_{\pi,w}$ )      # check whether  $\rho_{\pi,w} \in P$ 
```

This contradicts Theorem 7. □

**Example 16.** The following languages are all undecidable by Rice’s theorem:

- $\{\pi \mid \pi \text{ halts on all inputs}\};$
- $\{\pi \mid \pi \text{ halts on some input}\};$
- $\{\pi \mid \pi \text{ accepts the empty word}\};$
- $\{\pi \mid f_\pi = f_{\pi_0}\}$  for any fixed  $\pi_0$  — the set of programs equivalent to  $\pi_0$ .

**Remark 17.** There is one hidden detail in the proof of Rice’s theorem—the *construction* of the program  $\rho_{\pi,w}$  from  $\pi$  and  $w$ . For a proper reduction, we need this construction to be computable: given the code of  $\pi$  and the word  $w$ , we must be able to algorithmically produce the code of  $\rho_{\pi,w}$ . In Python (and any real programming language), this is obvious:  $\rho_{\pi,w}$  is just a piece of source code assembled by string concatenation. Formally, a programming language (‘encoding’) is *admissible* if it has an interpreter and ensures the ability to algorithmically hard-code arguments into a given program’s source code. Rice’s theorem holds for any admissible encoding, but can fail in artificial, pathological programming languages where programs cannot be algorithmically constructed (imagine, for example, a language where *somehow* every program has an additional flag—whether it halts on the empty input or not).

## 2.3 Enumerable languages

**Definition 18.** A language  $L \subseteq \Sigma^*$  is called **recursively enumerable** (or simply **enumerable**) if there exists a program that runs indefinitely and prints a (possibly infinite) list of words, such that:

- (1) every word printed by the program belongs to  $L$ ;
- (2) every word belonging to  $L$  is eventually printed by the program (possibly with repetitions).

**Example 19.** The set  $\text{EVEN} = \{n \in \mathbb{N} \mid n = 2k, k \in \mathbb{N}\}$  is enumerable:

```
def enum_even():
    x = 0
    while True:
        print(x)
        x = x + 2
```

**Example 20.** The language  $\{0, 1\}^*$  is enumerable:

```
def enum_bin_words():
    words = [""]      # create an array with just ε
    for w in words:
        print(w)
        words.append(w + "0")
        words.append(w + "1")
```

If we replace `print(w)` with `yield w`, `enum_bin_words()` becomes an infinite generator, allowing us to write `for w in enum_bin_words():` to iterate over all words in  $\{0, 1\}^*$  just like a Python `range`.

By analogy,  $\Sigma^*$  is enumerable and can be iterated through, for any alphabet  $\Sigma$ .

**Example 21.** The language  $\text{HALT} = \{\langle \pi, w \rangle \mid \pi \text{ halts on input } w\}$  is enumerable.

Since  $\{0, 1\}^*$  is enumerable, we can enumerate (binary encodings of) all programs and all input words. But to enumerate the pairs  $\langle \pi, w \rangle \in \text{HALT}$ , we would need to know if  $\pi$  halts on  $w$ . How can we tell, if HALT is undecidable?

Imagine we had infinitely many processors, one for each pair  $\langle \pi_i, w_j \rangle$ . Then, for every  $\langle \pi_i, w_j \rangle$  we can run a process  $p_{ij}$ , all of them in parallel, and wait. When a process terminates, we print the respective pair to the output. And those processes that never terminate will never have their pairs printed—that's exactly what we want.

The hard part is how to simulate infinitely many processes on a normal computer. We will rely on the following consideration. At the lowest level, a program does not execute all at once: its execution proceeds as a discrete sequence of elementary *machine instructions*—simple primitive steps: fetching a byte from memory, performing an arithmetic operation, or jumping to another location in code. Each instruction corresponds to a single unit of execution time.

So we can simulate the ‘parallelisation’ as follows. First, we arrange the infinite two-dimensional grid of processes  $p_{ij}$  into a single linear sequence:  $p_0, p_1, p_2, \dots$  (for instance, ordering by the sum  $i + j$ ). Then, we share the processor by gradually allocating more instructions to more processes:

- On step 0, run  $p_0$  for 1 instruction.
- On step 1, run  $p_0$  for a 2nd instruction, and  $p_1$  for 1 instruction.
- On step 2, run  $p_0$  for a 3rd instruction,  $p_1$  for a 2nd instruction, and  $p_2$  for 1 instruction.
- In general, on step  $s$ , run  $p_i$  for its  $(s - i + 1)$ -th instruction for each  $i \in \{0, 1, \dots, s\}$ .

Our program `enum_halt` will maintain a list of running processes and for each store their current progress. It will iterate through this list, pushing each process forward by one step, and appending newly launched processes. When a process terminates, the respective pair  $\langle \pi_i, w_j \rangle$  is printed out.

This technique has a vintage name, *dovetailing*. The metaphor comes from carpentry: a *dovetail joint* joins two boards by interlocking alternating fingers of wood. Here, we similarly interleave small slices of execution from different processes.

**Theorem 22** (Post, 1944). *A language  $L \subseteq \Sigma^*$  is decidable if and only if both  $L$  and its complement  $\bar{L} = \Sigma^* \setminus L$  are enumerable.*

*Proof.* ( $\Rightarrow$ ) Suppose  $L$  is decided by a program `decide_L`. Let `enum_words()` be a generator for  $\Sigma^*$ , analogous to the `enum_bin_words` from Example 20. We can enumerate  $L$  simply by filtering:

```
def enum_L():
    for w in enum_words():          # iterate over all words in  $\Sigma^*$ 
        if decide_L(w) == 1:
            print(w)
```

For the complement  $\bar{L}$ , we do the exact same thing, replacing `== 1` with `== 0`.

( $\Leftarrow$ ) Let `enum_L` and `enum_not_L` enumerate  $L$  and  $\bar{L}$ , respectively. On input  $w$ , run `enum_L` and `enum_not_L` in parallel until one of them outputs  $w$  (on two processors, or on one, via dovetailing). If `enum_L` outputs  $w$ , return 1; if `enum_not_L` outputs  $w$ , return 0. Since  $w$  must belong to either  $L$  or  $\bar{L}$  (and never both), exactly one will output  $w$  at some point, so this decides  $L$ .  $\square$

**Corollary 23.** *The complement of the Halting Problem,*

$$\overline{\text{HALT}} = \{\langle \pi, w \rangle \mid \pi \text{ does not halt on } w\}$$

*is **not** enumerable.*

## Appendix A. Proof by cases

Consider the following statement:

*For every integer  $n$ , the number  $n^2 + n$  is even.*

It has a simple proof:

It can be proven by considering two cases, depending on the parity of  $n$ .

- **Case:  $n$  is even.** Write  $n = 2k$ . Then  $n^2 + n = 4k^2 + 2k = 2(2k^2 + k)$ , which is even.
- **Case:  $n$  is odd.** Write  $n = 2k + 1$ . Then  $n^2 + n = (2k + 1)^2 + (2k + 1) = 4k^2 + 6k + 2 = 2(2k^2 + 3k + 1)$ , which is even.

In both cases,  $n^2 + n$  is even. Since every integer is either even or odd, the statement holds for all  $n$ .

Now an example from computability theory—Exercise 3(b) of Exercise sheet 2:

*If  $A, B \subseteq \mathbb{N}$  are decidable, then the language  $C = \{x \cdot y \mid x \in A, y \in B\}$  is decidable.*

Again, we can split the proof into cases depending on whether  $A$  or  $B$  is empty—analogous to splitting on the parity of  $n$  above.

- **Case:  $A = \emptyset$  or  $B = \emptyset$ .** Then there are no pairs  $(x, y) \in A \times B$  to multiply, so  $C = \emptyset$ . The empty language is decidable:

```
def decide_C(z):
    return 0
```

- **Case: both  $A$  and  $B$  are non-empty.** For any  $z > 0$ , there are only finitely many pairs  $(x, y)$  with  $x \cdot y = z$ . We test all factor pairs using the deciders for  $A$  and  $B$ :

```
def decide_C(z):
    if z == 0:
        return 1 if (decide_A(0) or decide_B(0)) else 0
    for (x, y) in factors(z):
        if decide_A(x) and decide_B(y):
            return 1
    return 0
```

A deciding program exists in each case, so  $C$  is decidable.

Note that the case split is a *proof method*: we do not need to provide a unique program that would somehow decide which is the case and behave accordingly. We only need to show that for every possible case in which  $A$  and  $B$  are decidable there exists a program (tailored to that case) that decides  $C$ . A single program that would take the program code of `decide_A` and `decide_B` and tell *which is the case* is impossible, by Rice's theorem.